

Passaggio di parametri:

→ parametri come "input" della procedura che permettono una comunicazione tra chiamante e chiamato

void fun(int x, int y)
tipo/valore ritorno parametri formali (nella def. della procedura)

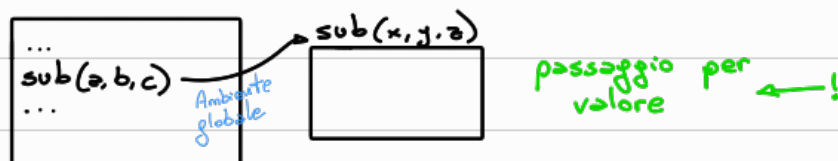
fun(2, 5) x...
parametri: y...
attuali fun(x+1, y+1) Ambiguità:
fun(z, w)

Aspetti progettuali del passaggio di parametri:

- 1 Modalità (valore, riferimento, risultato, valore-risultato)
- 2 Associazione parametri attuali e formali (ordine, in base al nome)

Modalità di passaggio di parametri: direzioni:

In-mode: chiamante → chiamato



Out-mode: chiamato → chiamante



Inout-mode: chiamante ↔ chiamato



Passaggio di parametri per valore

[0:22:00]

Il valore del parametro attuale (eventualmente ottenuto come valutazione di una espressione) viene passato come valore iniziale del corrispondente parametro formale

→ implementato creando una copia del valore attuale nel formale

fun ($x+1$, $y+2$)

il parametro attuale è un r-value
(quello che possiamo scrivere a dx di un assegnamento)

permette una certa versatilità nella scrittura dei parametri attuali

Svantaggio: no utilizzo della memoria

```
void foo (int x) { x=x+1; }  
:  
y = 1;
```

foo(y+1); → chiama a foo alloca x nell'ambiente locale di foo

→ alla chiamata $y+1$ viene valutato (2) e il valore viene copiato nella cella riferita da x

⇒ non c'è nessun legame tra x e y

→ alla terminazione di foo il RbA contenente x viene distrutto perdendo tutte le modifiche fatte a x

⇒ questa metodologia non permette di comunicare al chiamante attraverso i parametri:

Esempio

```
void swap (value int a, value int b) {  
    int tmp = a;  
    int a = b;  
    int b = tmp;  
}
```

int c = 3;

int d = 4;

swap(c, d);

esecuzione main

c → [3] d → [4]

chiamata di swap copia

a → [3] b → [4] tmp → []

esecuzione di swap

a → [4] b → [3] tmp → [3]

terminazione di swap

memoria: c → [3] d → [4]

rimane inalterata

eliminazione RbA di swap a → [4] b → [3] tmp → [3]

[0:40:00]

Passaggio di parametri per riferimento

→ viene passato un cammino di accesso al parametro formale In-out ✓

Pro: uso efficiente della memoria

Contro: creazione di alias

=> i parametri attuali possono essere solo l-value (riferimenti)

```
void foo (reference int x) { x=x+1; }  
:  
y=1;  
foo(y);
```

→ alla chiama si crea RdA con cella per x

→ Alla chiamata x diventa un riferimento alla cella riferita da y



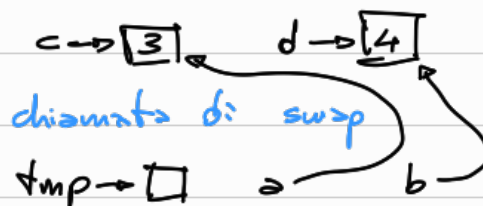
→ Alla distruzione viene distrutto RdA contenente il riferimento di x

=> tutte le trasformazioni fatte a x rimangono riferibili da y

Esempio

```
void swap (ref int a, ref int b) {  
    int tmp = a;  
    int a = b;  
    int b = tmp;  
}  
  
int c = 3;  
int d = 4;  
swap(c, d);
```

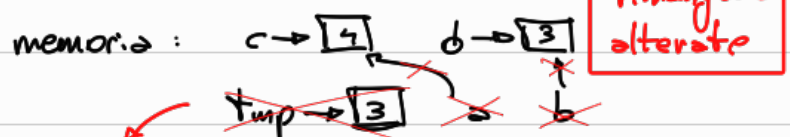
Esecuzione main



esecuzione di swap

tmp → [3] c → [4] ← a d → [3] ← b

terminazione di swap



eliminazione
del RDA di swap

[0:56:00]

[0:59:38]

[Esercizio nelle scan]

Esercizio 2

```
void pippo (int a, int b) {  
    a = 5+b; locale  
    b = 3a;  
}  
  
void main () {  
    int x=3, y=2;  
    pippo (x, x);  
}
```

Passaggio per valore

Esecuzione main

RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$

chiamata pippo

RdA pippo: $a \rightarrow [3]$ $b \rightarrow [3]$

esecuzione pippo

main: $x \rightarrow [3]$ $y \rightarrow [2]$

pippo: $a \rightarrow [8]$ $b \rightarrow [24]$

terminazione pippo

main: $x \rightarrow [3]$ $y \rightarrow [2]$

~~pippo~~

I valori nel main di x e y rimangono invariati:

Passaggio per riferimento

Esecuzione main

$x \rightarrow [3]$ $y \rightarrow [2]$

chiamata pippo

main $x \rightarrow [3]$ $y \rightarrow [2]$

pippo $a \rightarrow [3]$ $b \rightarrow [3]$

esecuzione pippo

main $x \rightarrow [3]$ $y \rightarrow [2]$

pippo $a \rightarrow [3]$ $b \rightarrow [3]$

terminazione pippo

main $x \rightarrow [24]$ $y \rightarrow [2]$

~~pippo~~

Alla terminazione $x=24$, $y=2$

Esercizio 3 (da completare)

```
void fun(int first, int second) {  
    first += first;  
    second += second;  
}
```

```
void main () {  
    int list[] = {3, 5};  
    fun(list[0], list[1]);  
}
```